AMAN SHUKLA

Class: FY-B.Tech (AIML) -1
Subject: Programming in C++-Theory
Subject Code: 25CS101
Seat: 2503215300025
PRN:
Attempt: Regular
Max Marks: 40.00
Obtained Marks: 30.00

Question History

Q#	Question	Marks	Q#	Question	Marks
1	1B 25 Mar, 7:20 am	0/1	3	3B 25 Mar, 7:19 am	3/3
1	1D 25 Mar, 7:20 am	0/1	3	3A 25 Mar, 7:18 am	3/3
1	1E 25 Mar, 7:20 am	0/1	4	4A 25 Mar, 7:20 am	0/4
1	1H 25 Mar, 7:20 am	1/1	4	4B 25 Mar, 7:17 am	4/4
1	1G 25 Mar, 7:20 am	1/1	5	5A 25 Mar, 7:20 am	0/4
1	1F 25 Mar, 7:20 am	1/1	5	5B 25 Mar, 7:17 am	4/4
1	1C 25 Mar, 7:19 am	1/1	6	6B 25 Mar, 7:20 am	0/4
1	1A 25 Mar, 7:19 am	1/1	6	6A 25 Mar, 7:17 am	4/4
2	2B 25 Mar, 7:20 am	0/3	7	7B 25 Mar, 7:20 am	0/4
2	2C 25 Mar, 7:18 am	3/3	7	7A 25 Mar, 7:18 am	4/4
2	2A 25 Mar, 7:18 am	2/3	8	8A 25 Mar, 7:20 am	0/4
3	3C 25 Mar, 7:20 am	0/3	8	8B 25 Mar, 7:20 am	0/4

Section-C

4.

(b) Algorithm to check whether a number is prime or not:

Step 1: Start.

Step 2: Input an integer (num) & initiate an integer type variable, $i = 3$.

Step 3: If num % 2 == 0, then print "Not a prime number"

~~Step 4: Else if num % i == 0 then print "Not a prime number".~~

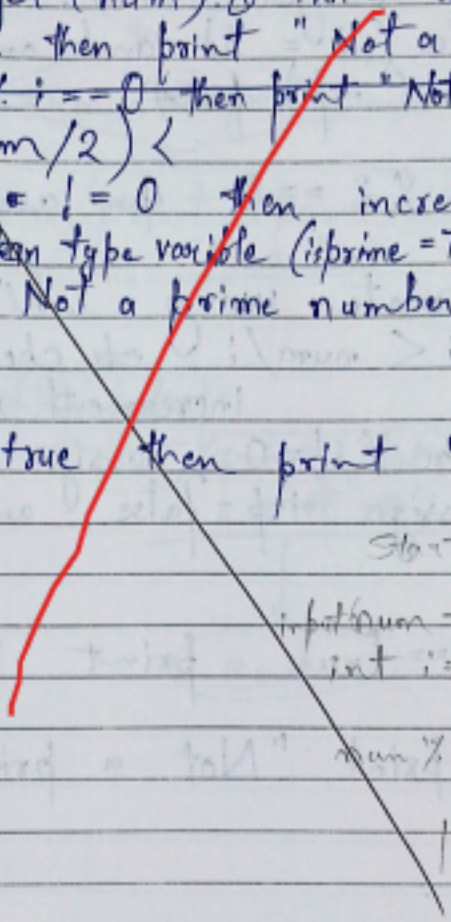
Step 4: While ($i <$ num/2) {

Step 5: If num % i != 0 then increment i by 1 (i.e, and initiate boolean type variable (isprime = True) ~~& go back to step 4~~)

Step 6: Else print "Not a prime number" and jump to step 8.

Step 7: If isprime == true then print "It is a prime number".

Step 8: Stop / end



Section C

4.(b) Algorithm :

Step 1 Start

Step 2 Input an integer (num) ; initiate an integer type variable (i = 3) and a boolen type variable (isp) = false

Step 3 If num % 2 == 0 then assign isp = false and jump to step 9.

Step 4 Else iterate using while loop :
while i < num/2 check :

Step 5 If num % i != 0 assign isp = true & go back to step 4

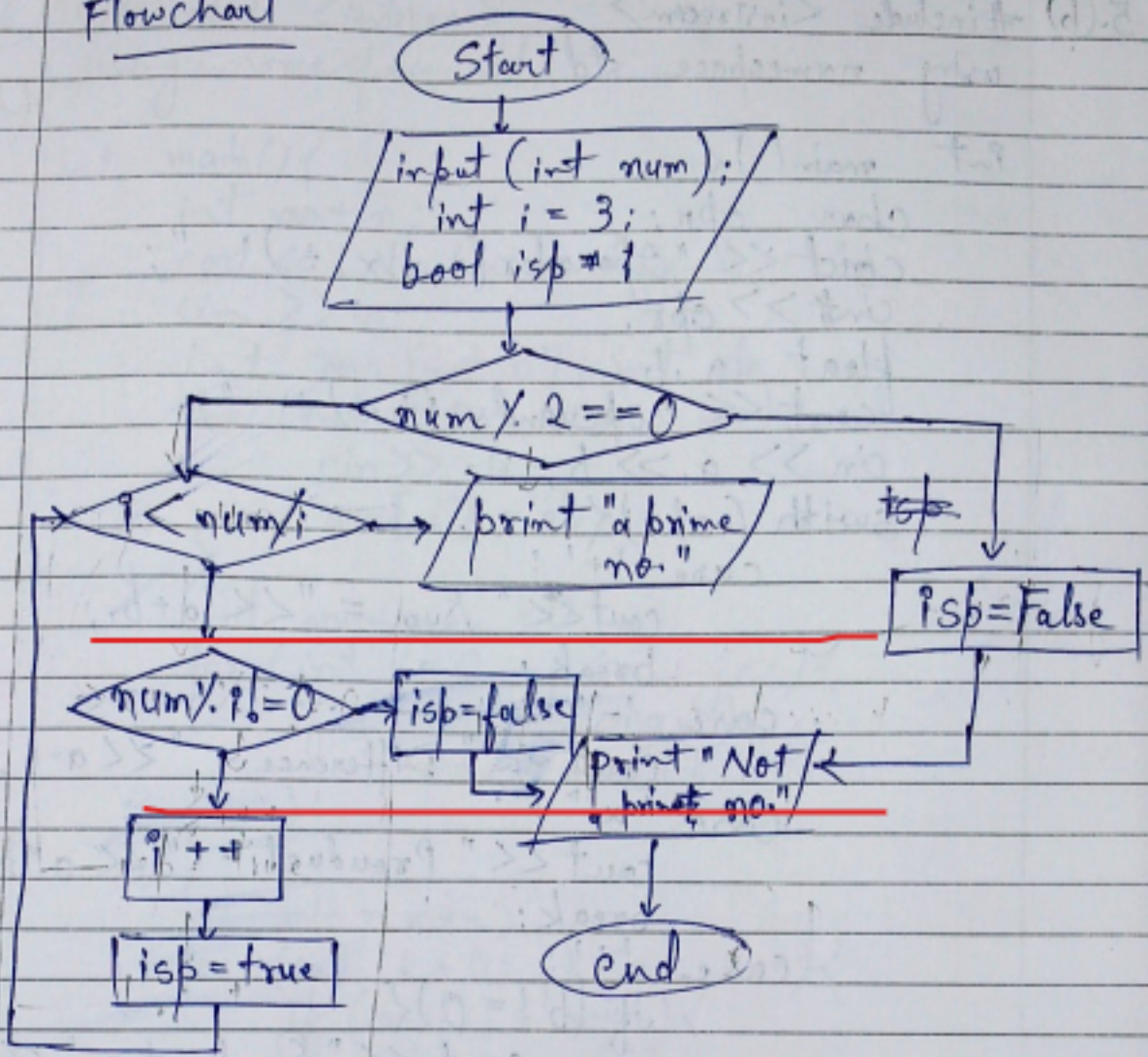
Step 6 Else assign isp = false and jump to step 7

Step 7 If isp == true print "A prime number"

Step 8 Else print "Not a prime number"

Step 9 end

Flowchart


Start
input (int num);
int i = 3;
bool isp = 1
num % 2 == 0
i < num/i
print "a prime no."
isp
isp=False
num % i != 0
isp=false
print "Not a prime no."
i ++
isp = true
end

5.(b)

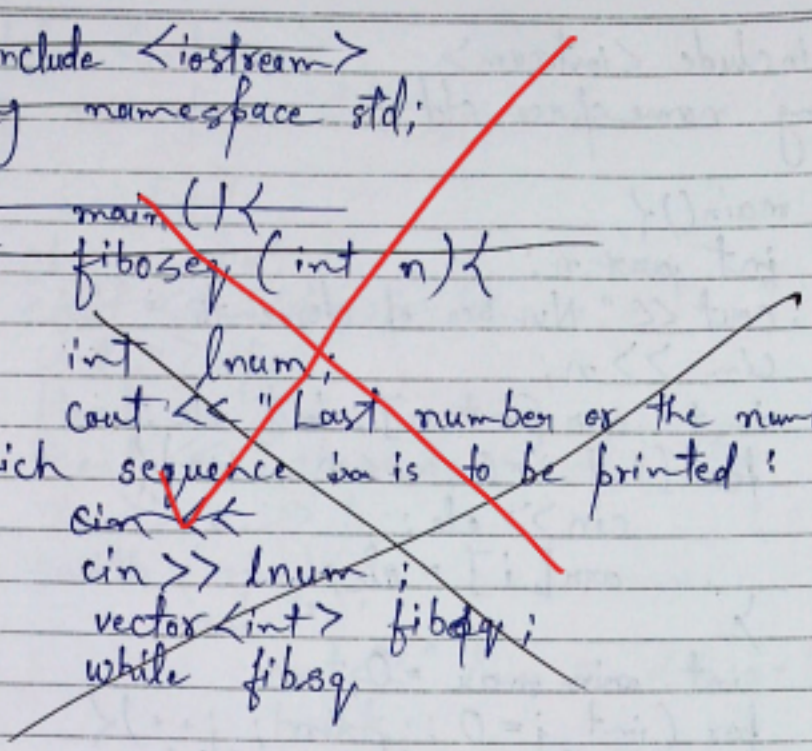
```
#include <iostream>
using namespace std;
int main(){
    char opr;
    cout << "Operator (+,-,x,÷) : ";
    cin >> opr;
    float a,b;
    cout << "operands (two) : ";
    cin >> a,>> b;
    swith (opr){
        case '+':
            cout<<" Sum = "<< a+b ;
            break;
        case '-':
            cout<<" Difference = " << a-b ; break;
        case 'x':
            cout << " Product = " << a*b;
            break;
        case '÷':
            if (b != 0){
                cout << " Quotient = " <<a/b;
            }else{
                cout << "Error! Division by Zero.";
            }
            break;
        default:
            cout << " Input data type error!";
            break;
    }
```

(6) (a)
```
#include <iostream>
using namespace std;
int main(){
    int n;
    cout << " Number of elements = ";
    cin >> n;
    int arr[n]; int ele;
    for (int i=0; i<n; i++){
        cin >> ele;
        arr[i] = ele;
    }
    int max = 0;
    for (int j=0; j<n; j++){
        if (arr[j] >= max){
            max = arr[j];
        }
    }
    int min = max;
    for (int k=0; k<n; k++){
        if (arr[i] < min){
            min = arr[i];
        }
    }
    cout << " largest number = " << max << " & smallest number = " << min;
```

(b)

7.

```
#include <iostream>
using namespace std;
int main(){
int fiboseq(int n){
    int lnum;
    cout << "Last number or the number upto which sequence in is to be printed : ";
    cin <<
    cin >> lnum;
    vector<int> fibseq;
    while fibseq
```



7.

```
#include <iostream>
using namespace std;
int fiboseq(int n){
    if (n==0){
        return 0;
    }else if (n==1){
        return 1;
    }else if (n>1){
        return fiboseq(n-1) + fiboseq(n-2);
    }
}
```

```
int main(){
    vector<int> fibsq; int i=0;
    int lnum;
    cin >> lnum;
    while (fibsq[fibsq.size()-1] <= lnum){
        fibsq.push_back(fiboseq(i));
        i++;
    }
    for (int j=0; j<fibsq.size(); j++){
        cout << fibsq[j] << " ";
    }
}
```

Q. (a)

Section B

2(a) For Standard library functions: These are the pre-defined, in-built functions which can be called using header files of C++ standard libraries (e.g., <iostream>, <cmath>, <string>, etc.) under the standard namespace using

E.g., cout, cin, abs(), sqrt(), length(), etc.

User-defined functions: These are the functions defined by the programmer which

can be called in the main() function as : function_name(parameter_list);

They are user built to perform specific tasks.

e.g., isprime(int n) fct is built to tell if n is a prime number or not.

(C) Recursion refers to the process of function calling itself until a base condition is reached.

```
int factorial(int n){
    if (n==1){                    // base condition
        return 1;
    }else if (n>1){
        return n* factorial(n-1); // recursive call
    }
}
```

e.g. for n = 5

stack recursion call stack = [5, 4, 3, 2, 1]

Q3. String representation in C++ can be through either c-style string which is an array of characters or std::string data type (dynamic list of characters including white-spaces)

C-style string ends in null-terminator (\0) while std::string is not.

for std::string, use header file <string>

E.g.,

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string str = "Hello";
}
```

- length operation (.length()) is used to gives the total number of elements in the string.

e.g., str.length() is 5.

- compare operation is used to compare two strings (i.e., if all corresponding characters are same or not)

e.g., ~~str2~~
string str2 = "hello"

~~str.compare(str2)~~ is False.

(b) To find a substring in a string, use find() function (in built).

e.g., ~~str.find("o")~~

int n = str.find("l")
initial

find() gives the index of the first occurence of the substring

in this case, n = 2.

To replace a substring in a string, use ~~replace() function~~.

Syntax:
~~str.~~replace(pos, n, subs)
starting
pos: index of the substring to replace

n: ~~no number~~ length of substring to replace

subs: ~~the substring replacing~~ the replaced ~~after~~ one.

e.g., str.replace(1, 1, "a"),

this gives str = "Hallo"

Section A

1. (a) In call by value, the value stored in the original variable is "copied" and the value of the value of orginal variable remains unchanged.

In call by reference, the formal parameter takes the is allocated to the memory location of original location and any changes made to that argument is reflected on the original variable.

(b)

(c)

```
return-type function-name(parameter-list){
    if (base condition){
        return (base value);
    }else{
        return (recursive call);
    }
}
```

d) Actual parameter is the argument passed while function calling.

Formal parameter is the variable defined while defining the function.

f) Character array ends with a null terminator (\0) whereas string object does not.

Character array has fixed size whereas string object has dynamic size.

g) Class is a user-friendly blueprint or prototype using which objects are made, with the help of defined member function and data members, without duplicating.

Objects are the elements which imitate real world objects

h) 1) Follows DRY (Don't Repeat Yourself) principle
2) Protects data from unauthorized access.

Section C

8.

(a)

$n=1$ b.c.

$x^n f(x)$